01 Overview & Architecture

Purpose — Raptorz Banking Earth is an autonomous, real-time fraud & financial-crime defense platform on **Gemini 3.5 + Google Cloud**. It reframes fraud defense from “block anything suspicious” to **verify the customer, block only real fraud**. A two-speed engine: a millisecond **fast path** (rules + Gemini Flash) clears ~90% of traffic; a seconds-deep **multi-agent path** (Google ADK + Gemini 3.5) investigates only the suspicious, then an adaptive, policy-aware decision.

Prerequisites

- Google Cloud project with billing linked (demo uses `ultra-mediator-506312-t2`, region `us-central1`).
- `gcloud` CLI authenticated; **Node.js 20+**; **Python 3.12+** (for local backend / eval).
- A Twilio account (for OTP + RM voice) and a Firebase project (for console sign-in).

Implementation

```
git clone https://github.com/TWILIGHTCLOUDCODERZ/Raptorz-AI-Builder-CUP.git
cd Raptorz-AI-Builder-CUP      # frontend/  backend/  infra/  merchant-site/  simulator/
```

Hero scenario: **Deepan**, a VIP resident of Kuala Lumpur (home country `MY`). A KL payment auto-approves; the same card used from Singapore is a cross-border anomaly → step-up OTP → block if not verified.

Validation — `npm --prefix frontend run build` and `python -m py_compile backend/app.py` both succeed.

Expected output — the reference architecture (below): ingress → Pub/Sub → fast path → deep multi-agent path → adaptive decision, with Twilio OTP step-up and the analyst console.

Troubleshooting — use Gemini `gemini-3.5-flash` on the `global` endpoint; `gemini-3.5-pro` returns 404 in this project. Keep Cloud Run in `us-central1`.

Security — proof-of-concept on **synthetic data only** — no real payments, card data or core-banking links. Every secret lives in Secret Manager; generative AI explains decisions, the policy/risk engine decides.

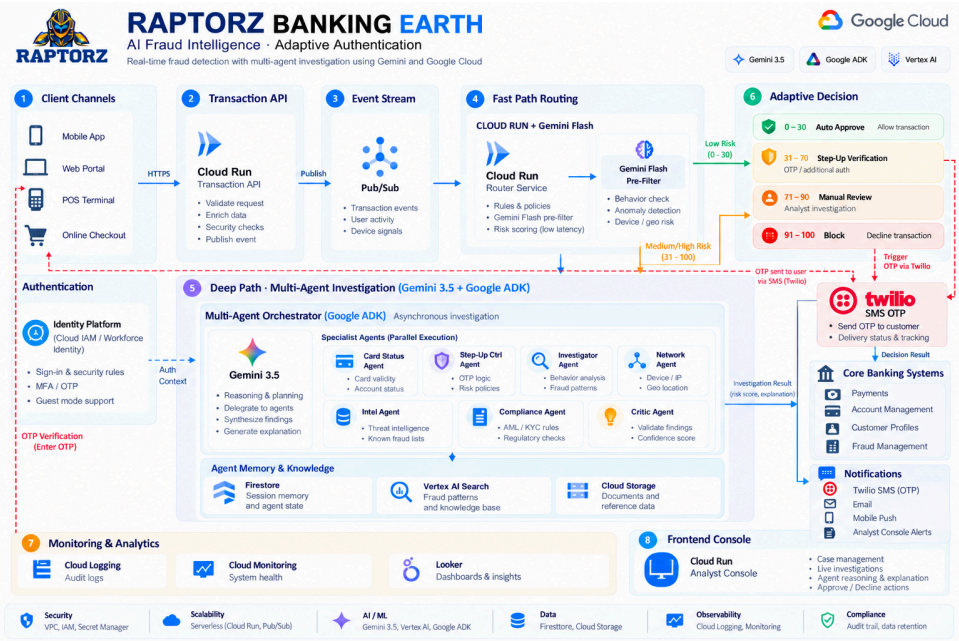


Figure 1 · Raptorz Banking Earth reference architecture (with Twilio OTP step-up).

02 Google Cloud Project & IAM Setup

Purpose — Provision the Google Cloud foundation: enable APIs, create the data stores and the event backbone, and create least-privilege service accounts the services run as.

Prerequisites

- Project Owner/Editor on the target project; billing linked.
- `gcloud` authenticated: `gcloud auth login` & `gcloud config set project <ID>`.

Implementation — one script enables APIs and creates Firestore, Pub/Sub topics, the Storage bucket, the Artifact Registry repo and the service accounts `raptorz-runtime` / `raptorz-sim`:

```
export PROJECT_ID=ultra-mediator-506312-t2
bash infra/setup.sh
```

APIs enabled: Vertex AI, Cloud Run, Pub/Sub, Firestore, Secret Manager, Cloud Build, Artifact Registry, Cloud Storage, Cloud Logging, Identity Toolkit.

Validation

```
gcloud services list --enabled --format="value(config.name)"
gcloud pubsub topics list          # transactions, transactions-dead-letter
gcloud firestore databases list    # (default) us-central1 FIRESTORE_NATIVE
gcloud iam service-accounts list   # raptorz-runtime@, raptorz-sim@
```

Expected output — the two topics, the Native-mode Firestore DB, and the two service accounts are listed; all APIs show as enabled.

Troubleshooting

- **Cloud Build 403 / permission denied on deploy** → grant the Cloud Build role to the compute SA: `gcloud projects add-iam-policy-binding $PROJECT_ID --member=serviceAccount:<project-number>-compute@developer.gserviceaccount.com --role=roles/cloudbuild.builds.builder`.
- **API not enabled** → re-run `setup.sh` or enable the named API manually.

Security — services run as dedicated, least-privilege SAs (`raptorz-runtime` for the app, `raptorz-sim` for the publisher) — never the default compute SA. Grant only Firestore, Secret Manager accessor, Pub/Sub and Vertex AI roles each needs.

03 Secrets & Configuration (Secret Manager)

Purpose — Keep every credential out of source and inject it at build/runtime from Secret Manager — the single source of truth.

Prerequisites — Secret Manager API enabled; the `raptorz-runtime` SA has `roles/secretmanager.secretAccessor`.

Implementation — three secrets:

- **firebase-config** — the frontend `.env` (Firebase web keys + `VITE_API_URL`): `PROJECT_ID=$PROJECT_ID bash infra/set-firebase-secret.sh`.
- **gemini-api-key** (optional) — only if not using Vertex AI via ADC: `PROJECT_ID=$PROJECT_ID GEMINI_API_KEY=... bash infra/set-gemini-secret.sh`.
- **twilio-config** — JSON used by OTP + RM voice:

```
printf '{"sid":"AC...","token":"...","from":"+1737...",
"to":"+9196...","verifyService":"VA..."}' > tw.json
gcloud secrets create twilio-config --data-file=tw.json   # or: versions add
gcloud secrets add-iam-policy-binding twilio-config \
  --member=serviceAccount:raptorz-runtime@$PROJECT_ID.iam.gserviceaccount.com \
  --role=roles/secretmanager.secretAccessor
rm tw.json
```

Validation — `gcloud secrets list` shows all three; `gcloud secrets versions access latest --secret=twilio-config` returns the JSON.

Expected output — secrets `firebase-config`, `gemini-api-key`, `twilio-config` listed with their create times.

Troubleshooting — **permission denied reading a secret at runtime** → add the accessor binding above. **bad JSON** → the server logs `[twilio] bad TWILIO_CONFIG json`; re-store valid JSON.

Security — never commit credentials. Mount secrets as env (`--set-secrets`), not baked into images. Rotate the Twilio auth token after any exposure and add a new secret version. Firebase *web* keys are public by design (they ship to the browser) — Secret Manager keeps them out of source; real protection is Auth rules + Authorized domains + API-key restriction.

04 Backend Deploy — Multi-Agent Service (Cloud Run)

Purpose — Deploy the FastAPI multi-agent investigation service to Cloud Run. It runs the agents on Gemini 3.5, exposes the investigation + case + step-up + RM endpoints, and persists cases to Firestore.

Prerequisites — `infra/setup.sh` done; secrets created; billing active.

Implementation — builds from source via Cloud Build (no local Docker); mounts the Gemini + Twilio secrets if present:

```
PROJECT_ID=$PROJECT_ID bash infra/deploy.sh
# service: raptorz-bfsi-backend  | model: gemini-3.5-flash  | location: global
```

Key endpoints: `/healthz`, `POST /investigate`, `POST /investigate/stream` (SSE), `GET /cases`, `GET /cases/{id}`, `/stepup/initiate`, `/stepup/resolve`, `/rm/*`.

Validation

```
B=$(gcloud run services describe raptorz-bfsi-backend --region us-central1 \
      --format='value(status.url)')
curl -s -X POST $B/investigate -H 'content-type: application/json' \
  -d @backend/sample_transaction.json | python -m json.tool
```

Expected output — JSON with `steps[]` (Orchestrator, Card Status, …, Critic) and `result` = `{decision, confidence, reason_codes, rationale}`.

Troubleshooting

- **Model 404** → set `GEMINI_MODEL=gemini-3.5-flash`, `GOOGLE_CLOUD_LOCATION=global`.
- **`/healthz` returns a Google 404 via some sandboxed `curl`** → harmless client quirk; `/investigate` and `/cases` work.
- **Cold start slow** → first request wakes the instance; scales to zero by design.

Security — demo uses `--allow-unauthenticated` for easy evaluation; set `ALLOW_UNAUTH=false` to require IAM auth. The service runs as `raptorz-runtime`; Vertex AI auth is via ADC (no key in code) unless `gemini-api-key` is mounted.

05 AI Agents — ADK + Gemini 3.5 Configuration

Purpose — Configure the reasoning agents. Each specialist is a Google ADK `LlmAgent` on Gemini 3.5; an Orchestrator fans out, and a policy-aware decision step enforces bank policy in code.

Prerequisites — `google-adk` + `google-genai` in `backend/requirements.txt`; Vertex AI reachable (ADC or key).

Implementation

- `backend/agents/adk_agents.py` — `run_agent()` wraps `LlmAgent` + `InMemoryRunner`; `APP_NAME="raptorz"`; `ADK_AVAILABLE` flag.
- `backend/agents/pipeline.py` — `investigate_stream()` emits each agent step: Orchestrator → Card Status → Step-Up Control → Investigator → Network Analyst → Intel → Compliance → Critic, then `_decide()`.
- **Policy gates in code** (`_decide`): stolen-card **hard block**; auto-pay + OTP step-up; **home-country geo gate** (country ≠ home → block unless `otp_verified` or memory explains).
- Decision agent emits **structured JSON** (`output_schema=DecisionResult`).

```
RAPTORZ_USE_ADK=true     # default; false forces the GenAI-SDK path
RAPTORZ_OFFLINE=true     # deterministic, no cloud calls (demos/tests/eval)
```

Validation — a step's `evidence` contains `engine=adk` (or `engine=genai-sdk` on fallback). Run offline: `RAPTORZ_OFFLINE=true uvicorn app:app --port 8080`.

Expected output — 7–8 agent steps with short rationales + a decision with reason codes and confidence.

Troubleshooting — **ADK import/SSL fails** → auto-falls back to the Google GenAI SDK, then to deterministic offline mode, so the demo never breaks. **Empty/garbled JSON** → the schema path retries; a mock decision is the last resort (`reason_codes` gains `llm_fallback`).

Security — **policy is enforced in code, never delegated to the model**. Model Armor guards prompt-injection/PII around Gemini calls. The LLM produces explanations only; the gate makes the block/approve call.

06 Frontend Console Deploy (Cloud Run)

Purpose — Deploy the React + TypeScript analyst console: Command Center, Investigations, Network Graph, Customer 360, Compliance/SAR, RM Workbench and the interactive Architecture explorer.

Prerequisites — backend deployed; `firebase-config` secret set; a Firebase project with Email/Password (and Google) sign-in.

Implementation

```
PROJECT_ID=$PROJECT_ID bash infra/deploy-frontend.sh
# service: raptorz-bfsi-frontend
# VITE_API_URL is auto-derived from the raptorz-bfsi-backend service URL
```

The script fetches `firebase-config`, injects the `VITE_FIREBASE_*` values into the build, and serves the static SPA on Cloud Run (nginx).

Validation — open the printed URL; **continue as guest** (no account) or sign in with Firebase. The Command Center and live feed render.

Expected output — the console at `/` and the executive walkthrough at `/#fraud-journey`.

Troubleshooting

- **"This domain is not authorized" on Google/email sign-in** → Firebase console → Authentication → Settings → Authorized domains → add the Cloud Run domain. (Guest login works without this.)
- **Blank API data** → confirm `VITE_API_URL` points at the live backend; check CORS (`ALLOWED_ORIGINS`).

Security — Firebase Auth (email/password + Google) + guest mode; web keys kept out of source via Secret Manager; restrict the Firebase API key to the app's domains + Identity Toolkit in Cloud Console → Credentials.

07 Merchant Demo — Earth Mart / Mars Mart

Purpose — A standalone storefront that checks out through Raptorz, demonstrating the home-country geo gate and the Card/UPI + OTP flow. Deployed as its own Cloud Run service.

Prerequisites — backend live; `twilio-config` secret (for the Singapore OTP); `raptorz-runtime` SA.

Implementation

```
PROJECT_ID=$PROJECT_ID bash infra/deploy-merchant.sh   # service: earth-mart
```

- **Earth Mart Malaysia** (auto-pay ON) → approved by the agents, no OTP. RM 450 headphones.
- **Mars Mart Singapore** (auto-pay OFF, cross-border) → Twilio Verify OTP. SGD MacBook.
- A **hidden store-location toggle** in a corner flips Malaysia / Singapore; checkout offers **Card or UPI**.
- Optional **VM + VPN** (`infra/setup-merchant-vm.sh`) runs in *detect mode* so the real egress IP decides the country.

Validation — `curl -s -X POST $M/pay -d '{"mode":"MY","method":"card"}'` returns `decision:"Approved"`; `mode:"SG"` returns `needsOtp:true`.

Expected output — a Card/UPI modal; Malaysia approves instantly; Singapore sends an OTP and shows a 60s countdown.

Troubleshooting — **"Failed to parse URL /pay"** (page opened as `file://`) → the client falls back to the backend; serve via the Cloud Run URL. **Geo not flipping on Cloud Run** → detect mode needs the VM + VPN; otherwise use the toggle.

Security — the merchant calls the backend **server-side**; no card data is stored (demo placeholders only). OTP codes live server-side in a short-TTL map; Twilio creds come from Secret Manager.

08 Scenario 1 — Home-Country Auto-Approve

Purpose — Show that a legitimate in-home-country payment is approved with zero friction — memory and policy recognise the customer instead of challenging them.

Prerequisites — merchant + backend + frontend live; customer **Deepan** seeded (home `MY`, tier VIP, confirmed-legit KL pattern).

Implementation

1. Open the merchant; keep the store on **Earth Mart Malaysia**.
2. Click **Pay** → choose **Card** or **UPI** → confirm.
3. Backend call: `/investigate` with `auto_pay:true, otp_verified:true, country:"MY"`.

Validation — in the console **Investigations** a new case appears for Deepan / Earth Mart Malaysia, decision **Approved**, ~8 agents, reason codes include `confirmed_legit_memory`.

Expected output — merchant shows “Payment approved”; the case rationale notes behaviour matches the local pattern; no OTP was requested.

Troubleshooting — **unexpectedly blocked** → confirm the customer’s `home_country` is `MY` and the confirmed-legit memory exists in Firestore (`customers/Deepan`). **No case in feed** → the feed polls `/cases` every ~4s; check the backend URL.

Security — auto-pay is an explicit customer enrollment (consent). The approve path still runs the full investigation and records an auditable case — frictionless is not unlogged.

09 Scenario 2 — Cross-Border Step-Up OTP (Twilio Verify)

Purpose — A cross-border payment is not cold-declined: the agents step up with a one-time passcode. Correct OTP → approved; wrong / none within 60s → the card is blocked.

Prerequisites — `twilio-config` with a Twilio **Verify** service (`verifyService=VA...`); customer mobile reachable.

Implementation

1. Switch the store to **Mars Mart Singapore**; click **Pay**.
2. Merchant `/pay` → backend `/stepup/initiate` opens a live **Step-up** case (agent *Step-Up Control*, status `running`, "OTP sent — waiting"), then Twilio Verify sends the SMS.
3. Enter the code → `/verify-otp` → `/stepup/resolve {case_id, otp_verified}` finishes the same case.

```
# Verify REST (what the server does)
POST verify.twilio.com/v2/Services/{VA}/Verifications      To, Channel=sms
POST verify.twilio.com/v2/Services/{VA}/VerificationCheck  To, Code  -> status=approved
```

Validation — the case flips **Step-up** → **Approved** (correct) or **Blocked** (wrong/timeout); reason codes include `geo_step_up_completed`/`otp_verified` or `geo_location_mismatch`/`otp_not_received`.

Expected output — WhatsApp/SMS "Your Raptorz verification code is NNNNNN"; merchant shows Approved or "card blocked"; the full story is visible one-by-one in Investigations.

Troubleshooting — **Twilio trial can't send freeform SMS/WhatsApp** (`ContentSid Required` / `572006`) → use **Twilio Verify** (templated, trial-safe); if Verify is unavailable the merchant shows an on-screen demo code. **Code mismatch** → ensure Verify (not a fixed sample template) is configured.

Security — OTP generated & validated by Twilio Verify (never compared in plaintext by us); 60-second window; a wrong code blocks the card and opens a fraud case. Customer mobile is masked in the UI.

10 Scenario 3 — RM Voice-Call Card Recovery

Purpose — For a blocked card, the Relationship Manager places an automated Twilio **voice call**. The AI agent explains the situation and unblocks the card **only after validated voice approval** (AutoPay stays disabled).

Prerequisites — `twilio-config` with `from` (a voice-capable Twilio number) and `to`; backend public URL for the TwiML webhook.

Implementation — open the console → **RM Workbench** (card shows **Blocked**) → **Initiate AI voice call**.

- `POST /rm/call` dials via Twilio (To/From/Url only on trial).
- `POST /rm/voice` returns TwiML: `<Gather input="speech dtmf">` reads the unblock script.
- `POST /rm/voice/collect`: **yes** / **press 1** → unblock + log; **no** / **2** → stay blocked; silence → stay blocked.

Card status + call logs persist on the Firestore customer doc; `GET /rm/customer` feeds the UI; `POST /rm/reset` re-arms the demo; `POST /rm/simulate` drives the same state machine when a live call can't ring.

Validation — on "yes" the card flips **Blocked** → **Unblocked** with an approval + unblock log; "no"/no-answer leaves it Blocked with the outcome logged.

Expected output — an answered call reading the Singapore-transaction script; the RM Workbench status and expandable activity log update live.

Troubleshooting — **Trial calls only reach *verified* numbers** (unverified → `no-answer`, duration 0); verifying is also blocked on trial (`10002`) → use the **demo simulate** buttons or upgrade. **"Invalid/disallowed parameters"** → send only `To/From/Url` (trial rejects StatusCallback). **411 on a body-less POST** → client artifact; browser `fetch`/Twilio are fine.

Security — the card **never** unblocks without an explicit validated "yes". Every call attempt, answer, decision and status is written to the audit log. AutoPay stays disabled; OTP is still required on future payments.

11 Investigations, Customer 360 & Live Agent Feed

Purpose — Give analysts a live, explainable view: every case streams its agent reasoning one-by-one, with a 360° customer profile and a draft SAR.

Prerequisites — backend live (Firestore-backed cases); frontend console reachable.

Implementation

- **Investigations** polls `GET /cases` (summaries) and loads `GET /cases/{id}` on select; the detail replays agents one-by-one (700ms reveal) then shows the decision + any block banner.
- **Live stream**: `POST /investigate/stream` emits SSE `event: step` then `event: case`.
- **Customer 360**: VIP card, 6 transactions with location + agent detail; phone/email **masked by default** with reveal-on-click.
- **Compliance** / **SAR**: auto-drafted case subject, risk, priority, regulator format.

Validation — run a merchant payment; within ~4s the new case appears in the feed and plays its agents; the decision matches the merchant result.

Expected output — a scrolling case list (two names, Approved + Blocked), a per-case agent timeline, and a Customer 360 card with masked contacts.

Troubleshooting — **feed empty** → verify the backend URL and that cases persist (`GET /cases` returns rows). **case stuck mid-animation** → the detail re-animates when a pending case grows (keyed on id + step count).

Security — PII (phone/email) is masked and only revealed on explicit click; every decision carries reason codes, confidence and an evidence chain for audit & model-risk governance.

12 Operations — Monitoring, Benchmark, Troubleshooting & Security

Purpose — Run Raptorz day-to-day: observe it, prove its accuracy, resolve common issues, and meet security/audit obligations.

Prerequisites — all services deployed; Cloud Logging/Monitoring enabled.

Implementation & Validation

- **Observe**: Cloud Logging (audit logs), Cloud Monitoring (health); per-service logs via `gcloud run services logs read <svc>`.
- **Benchmark**: `python backend/eval/evaluate.py` over 276 labeled txns.

Metric	Rules-only	Raptorz
Fraud caught (recall)	12.7%	95.2%
Hard false-decline rate	39.0%	3.8%
Block precision	8.8%	88.2%

Troubleshooting (quick reference)

Symptom	Fix
Model 404	gemini-3.5-flash on `global`
Cloud Build 403	grant `cloudbuild.builds.builder`
Sign-in domain error	add domain to Firebase Authorized domains
OTP not delivered	use Twilio Verify; trial blocks freeform
RM call no-answer	trial calls verified numbers only; use simulate

Security & Audit

- **Secrets** in Secret Manager only; rotate the Twilio token after exposure (`versions add` + redeploy). Keep keys out of git history (the Firebase key was purged from history).
- **Consent**: auto-pay & OTP are customer-authorised; the card unblocks only on validated approval.
- **Access control**: least-privilege SAs; set `ALLOW_UNAUTH=false` + IAM for non-demo.
- **Audit**: every decision (reason codes, confidence, evidence) and every RM action is logged and retained in Firestore / Cloud Logging for governance and regulatory review.

Proof-of-concept on synthetic data — no real payments, messages or banking integrations.